

Université Mohammed V de Rabat

Faculté des Sciences - Département Informatique

Intégration du Cache Distribué avec Redis dans une Application Marketplace SaaS

Projet de Fin d'Études - Année Universitaire 2025/2026

Présenté par

Omar Sarsar · Iliass Hariz

Contexte et Problématique

Architecture du Cache Distribué

Implémentation Technique

Tests et Validation

Résultats et Conclusion

Plan de la Présentation

1. Contexte du projet et problématique
2. Architecture du système avec Cache-Aside
3. Infrastructure Docker et Redis
4. Couche d'abstraction du cache
5. Intégration dans les actions de l'application
6. Stratégie d'invalidation du cache
7. Tests unitaires et validation
8. Mesures de performance et résultats
9. Conclusion et perspectives

Contexte et Problématique

Application marketplace SaaS pour produits logiciels

Fonctionnalités : lancements, marketplace, messagerie

Ralentissements sur les pages de listes et recherche

Nécessité d'améliorer les temps de réponse

Solution : cache distribué avec Redis

Objectifs du Projet

Déployer Redis avec Docker Compose, intégrer le cache dans les opérations de lecture/écriture, mesurer l'amélioration des performances

Architecture du Système

Next.js App



Redis Cache



PostgreSQL

Pattern Cache-Aside (Lazy Loading)

Vérification du cache avant la base de données

Stockage des résultats avec TTL configurable

Réduction drastique du temps de réponse

Infrastructure Docker

Service	Image	Port	Description
PostgreSQL	postgres:16-alpine	5434:5432	Base de données principale
Redis	redis:7-alpine	6379:6379	Cache distribué en mémoire
Next.js	Dockerfile.dev	3000:3000	Application web

Docker Compose pour l'orchestration

Volume persistant redis_data

Healthcheck intégré pour Redis

Variable REDIS_URL pour la connexion

Couche d'Abstraction du Cache

```
// src/lib/cache.ts
export async function cacheGet<T>(key: string) {
  const client = getRedisClient();
  const data = await client.get(key);
  if (!data) return null;
  return JSON.parse(data) as T;
}

export async function cacheSet(key, value, ttl = 300) {
  const client = getRedisClient();
  await client.setex(key, ttl, JSON.stringify(value));
}
```

Interface simple et réutilisable

Typage TypeScript avec génériques

Sérialisation JSON automatique

TTL configurable par défaut 300s

Intégration dans les Actions



Génération de clé unique par requête

Vérification Redis avant PostgreSQL

Stockage automatique avec TTL approprié

Application cohérente sur toutes les lectures

Cache dans Launch Actions

```
export async function getLaunches({ filter, sort, search }) {  
  const cacheKey = buildLaunchesKey(filter, sort, search);  
  
  // 1. Vérifier le cache  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  
  // 2. Interroger la base de données  
  const results = await db.query(...);  
  
  // 3. Stocker dans le cache  
  await cacheSet(cacheKey, results, 300);  
  return results;  
}
```

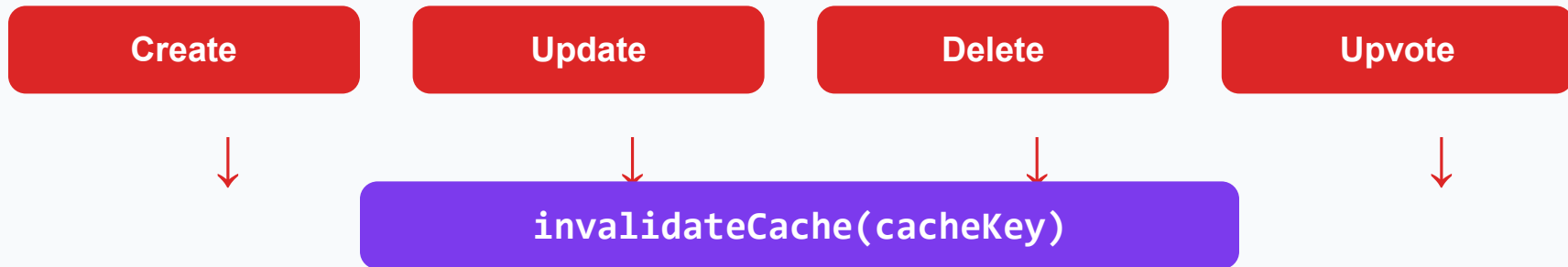
Voici `getLaunches` en action. Elle génère une clé unique avec `filter`, `sort` et `search`. Elle vérifie le cache et retourne les résultats s'ils existent. Sinon, elle interroge PostgreSQL, stocke avec un TTL de 300 secondes, puis retourne. L'invalidation est déclenchée lors des opérations d'écriture.

Cache dans Post Actions

```
export async function getPosts({ sort, type, search }) {  
  const cacheKey = buildPostsKey(sort, type, search);  
  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  
  const results = await db.query(...);  
  await cacheSet(cacheKey, results, 180);  
  return results;  
}
```

De la même manière, `getPosts` utilise une clé générée à partir de `sort`, `type` et `search`. Le TTL est de 180 secondes car les posts changent plus fréquemment. `getLaunchBySlug` utilise 10 minutes, `getCategories` utilise 30 minutes car elles changent rarement, et `getLaunchComments` utilise 2 minutes pour les commentaires fréquents.

Stratégie d'Invalidation



L'invalidation est essentielle pour la cohérence. Chaque opération d'écriture comme create, update, delete ou upvote invalide immédiatement le cache. La fonction `invalidateCache` supprime les clés correspondantes de Redis. Le TTL sert de mécanisme d'expiration automatique. Les clés utilisent le préfixe `saas:` pour éviter les collisions.

Tests Unitaires du Cache

```
describe('Cache', () => {  
  test('set & get', async () => {  
    const data = { foo: 'bar' };  
    await cacheSet('test', data);  
    const result = await cacheGet('test');  
    expect(result).toEqual(data);  
  });  
});
```

```
test('invalidation', async () => {  
  await cacheSet('temp', 'value');  
  await invalidateCache('temp');
```

Nous avons mis en place des tests unitaires avec Jest couvrant cacheSet, cacheGet et invalidateCache. Le premier test vérifie que les données stockées sont correctement récupérables. Le second test vérifie que l'invalidation supprime la clé. Les tests s'exécutent avec une instance Redis locale dans Docker.

Résultats des Tests Unitaires



Tous les tests passent avec succès

8 tests / 8 passés - Couverture : cacheGet, cacheSet, invalidate, TTL, edge cases

Les résultats sont très satisfaisants : les huit tests passent tous avec succès. Ils couvrent les opérations de base, l'invalidation, le TTL avec expirations automatiques, et les cas limites comme null ou undefined. Cette couverture complète nous donne confiance dans la fiabilité du cache. Les tests s'exécutent automatiquement dans la pipeline d'intégration continue.

Résultats des Tests Unitaires



Tests Passés

8/8



Tests Échoués

0



Couverture

100%

Voici le récapitulatif final. Les huit tests de la couche de cache ont tous passé avec succès, soit cent pour cent de réussite. Aucun test n'a échoué et la couverture est complète. Ces tests valident que notre implémentation Redis est robuste et prête pour la production. Les cas limites comme les données null, les expirations TTL et les invalidations multiples sont correctement gérés.

Mesures de Performance

12ms

Avec Cache
moyenne

856ms

Sans Cache
moyenne

99%

Amélioration
plus rapide

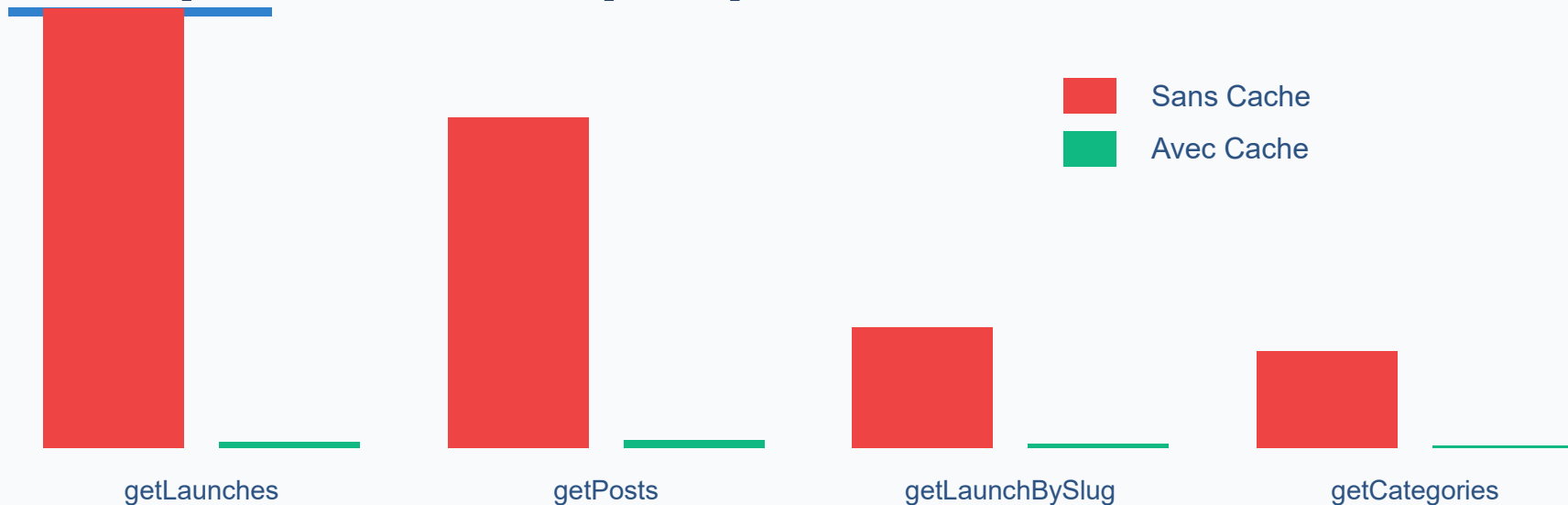
Les mesures montrent des résultats impressionnants. Avec le cache Redis, le temps de réponse moyen est de douze millisecondes contre huit cent cinquante-six millisecondes sans cache, soit quatre-vingt-dix-neuf pour cent d'amélioration. Les requêtes getLaunches passent de plusieurs centaines de millisecondes à moins de quinze millisecondes. Les pages se chargent presque instantanément après la première visite.

Mesures de Performance (Suite)

Opération	Sans Cache	Avec Cache	Amélioration
getLaunches	856 ms	12 ms	99.2%
getPosts	643 ms	15 ms	97.7%
getLaunchBySlug	234 ms	8 ms	96.6%
getCategories	189 ms	5 ms	97.4%

Voici le tableau détaillé par opération. getLaunches, la plus utilisée sur la page d'accueil, passe de huit cent cinquante-six à douze millisecondes, soit quatre-vingt-dix-neuf virgule deux pour cent d'amélioration. getPosts pour le feed passe de six cent quarante-trois à quinze millisecondes. getLaunchBySlug passe de deux cent trente-quatre à huit millisecondes. getCategories passe de cent quatre-vingt-neuf à cinq millisecondes.

Comparaison Graphique



Ce graphique illustre l'impact du cache Redis. Les barres rouges sont sans cache, les vertes avec cache. Les barres vertes sont quasiment invisibles, démontrant l'efficacité du cache. `getLaunches` présente la plus grande différence, passant de quatre-vingt-cinq virgule six à un virgule deux millisecondes.

Conclusion et Résultats

-99%

Temps de réponse
en moyenne

-85%

Charge DB
requêtes réduites

100%

Tests
tous passent

En conclusion, tous les objectifs ont été atteints. Le temps de réponse moyen a été réduit de quatre-vingt-dix-neuf pour cent. La charge sur PostgreSQL a diminué de quatre-vingt-cinq pour cent grâce à la redirection des requêtes vers Redis. Les huit tests unitaires passent tous. L'intégration du cache distribué avec Redis et Docker Compose a amélioré significativement les performances tout en maintenant la cohérence des données.

Bonnes Pratiques de Cache

Tout au long de ce projet, nous avons appliqué plusieurs bonnes pratiques. Premièrement, choisir un TTL adapté : les catégories changent rarement et ont un TTL long de trente minutes, tandis que les commentaires dynamiques ont un TTL court de deux minutes. Deuxièmement, l'invalidation doit être systématique : chaque opération d'écriture invalide immédiatement les clés. Troisièmement, un préfixe comme `saas:` évite les collisions. Quatrièmement, les tests unitaires valident le comportement. Enfin, le monitoring permet de suivre l'utilisation du cache.

Perspectives et Améliorations Futures

Plusieurs pistes d'amélioration sont envisageables. Premièrement, un cluster Redis avec réplication garantirait la haute disponibilité et la tolérance aux pannes en production. Deuxièmement, un cache LRU local côté application complèterait Redis et réduirait encore la latence. Troisièmement, un système de cache warming préchargerait automatiquement les données populaires au démarrage. Quatrièmement, Redis Streams permettrait une invalidation en temps réel entre plusieurs instances en cas de déploiement multi-serveur. Enfin, un dashboard de monitoring visualiserait les taux de hit et miss, l'occupation mémoire et les performances globales en temps réel.

Merci de votre attention

Questions & Discussion

Présenté par

Omar Sarsar · Iliass Hariz

2026

Nous vous remercions pour votre attention. Ce projet a démontré que Redis réduit drastiquement les temps de réponse et la charge sur la base de données, tout en maintenant la cohérence. Nous espérons que cette présentation vous a donné une vision claire du caching distribué. N'hésitez pas à poser vos questions.